

Acciones sobre todos los elementos de arrays: Método `map()`

`array.map()` Es un método muy parecido a método `forEach()`, pero más cómodo de utilizar si se desea crear un nuevo array a partir del original. Sintaxis:

```
array.map(callback)
```

`callback` es la función que se ejecutará para cada uno de los elementos del array (igual que el `callback f1` del ejemplo anterior de uso de `forEach`).

`callback` recibe 3 argumentos iguales a `forEach`.

La diferencia entre `forEach` y `map` es que `map` retorna un nuevo array, siendo cada elemento de este nuevo array el valor devuelto por la función `callback`. Esto se puede hacer también con `forEach` pero con `forEach` es necesario crear el nuevo array e ir incorporando el valor deseado en cada ejecución del `callback`.

Otra ventaja de `map` es que devuelve directamente un nuevo array, por lo que nos permite ir encadenando llamadas a métodos de array.

```
function callback (item,index,arr) {return item.toUpperCase() }  
//cada elemento devuelto se incorporará al nuevo array  
let sistemaSolarMayus=sistemaSolar.forEach(f1) ;  
//sistemaSolarMayus será ["MERCURIO", "VENUS", "TIERRA", ...]
```

Con `forEach` habríamos necesitado hacer lo siguiente:

```
let sistemaSolarMayus=[]  
function callback(elem){sistemaSolarMayus.push(elem.toUpperCase()) }  
sistemaSolar.forEach(f1)
```


Acciones sobre todos los elementos de arrays: Método `filter()`

`array.filter()` funciona como el método `map()`, ejecuta un `callback` por cada elemento del array, y retorna un nuevo array, pero en este caso, filtrado.

```
array.filter(callback)
```

`callback` recibe 3 argumentos iguales a `forEach` y `map`.

`callback` es la función que se ejecutará para cada uno de los elementos del array, igual que `map`, con la diferencia de que solo insertará el dato en el nuevo array si éste no es `false`. Esto no se puede conseguir con el método `map`, ya que si `map()` devuelve `false` el dato incluido en el nuevo array será `false`.

Ejemplo: a partir del array anterior, `sistemaSolarMayus`, creamos un nuevo array en el que estén incluidos solo los planetas que no tienen una "U" en su nombre.

```
function callback (item,index,arr) {  
  if (item.indexOf("U")==-1)  
    return item  
  else  
    return false  
}  
let sistemaSolarMayusU=sistemaSolarMayus.filter(callback);  
//sistemaSolarMayusU será ["TIERRA, "MARTE"]
```

Se puede escribir también de esta forma más breve:

```
let sistSolarMayusU=sistSolarMayus.filter(pl=>pl.indexOf("U")==-1)
```


Acciones sobre todos los elementos de arrays: Método `reduce()`

`array.reduce()` procesa todos los elementos de un array para reducirlos a un solo valor.

`array.reduce(callback, valorInicial)`

`callback` recibe 4 argumentos que son:

1. El valor del elemento anterior: en la primera vuelta no existe este valor, por lo que al llamar a `reduce` se le puede indicar, de forma opcional, además de la función `callback` un valor inicial que es el que será el valor del elemento anterior en la primera vuelta.
2. El valor del elemento actual.
3. El índice actual
4. El propio array original.

Ejemplo: a partir del array, `[4, 3, 2, 5]`, calcula la suma de todos los números del array.

```
function callback (previo, actual, index, arr) {  
    return previo+actual  
}
```

```
let total=[4,3,2,5].reduce(callback,1);  
//total será 15
```

Se puede escribir también de esta forma más breve:

```
let total=[4,3,2,5].reduce((previo, actual)=>previo+actual,1)
```


Acciones sobre los elementos de arrays: Método `every()` `some()`

`array.every()` comprueba si todos los elementos del array hacen que la función pasada como callback devuelva `true`.

```
array.every(callback)
```

callback recibe 3 argumentos iguales a `forEach`, `map` y `reduce`.

callback debe devolver `true` o `false`.

Ejemplo: a partir del array `sistemaSolarMayus`, calcula si todos los planetas contienen la vocal "U".

```
function callback (elem,index,arr) {  
  return (elem.indexOf("U") !== -1) ? true : false }  
let respuesta=sistemaSolarMayus.every(callback) //devuelve false
```

Ejemplo: Elimina del array los planetas que no contengan la vocal "U" y vuelve a ejecutar.

Si se quiere comprobar si algún elemento cumple, se puede utilizar el método `some()`

```
let respuesta=sistemaSolarMayus.some(callback) //devuelve true
```

`array.find()`: devuelve el valor del primer elemento del array que hace que la función callback devuelva `true`.

```
array.find(callback)
```

callback recibe 3 argumentos iguales a `forEach`, `map`, `every` y `some`.

Ejemplo: Devuelve del array `sistemaSolarMayus` el primer planeta que contenga la vocal "A".

Ejercicio métodos `map`, `filter`, `reduce`, `every`, `some`, `find`

Tenemos un array de objetos persona. Cada objeto persona guarda de cada persona el nombre, provincia de nacimiento y fecha de nacimiento, y sexo.

1. Crea un nuevo array en el que aparezcan los siguientes datos de cada persona: nombre y edad y luego muéstralo.
2. Crea un nuevo array en el que solo aparezcan las personas de sexo Masculino y luego muéstralo.
3. Calcula la cantidad total de años que tienen entre todas las personas.
4. Indica si todas las personas son mujeres.
5. Indica si alguna persona es de Valencia.
6. Devuelve el nombre y el sexo de la primera persona que es de Alicante.

